

Graph Theory

– Course 3 –

Chapter 3 Graph Coloring

Introduction

Graph coloring problems arise in many contexts:

Example 1: the scheduling of university exam sessions. If V represents the set of courses and an edge connects two courses that have at least one student in common, the chromatic number is the minimum number of periods required to schedule exams so that every student can take all its exams.

Example 2: the coloring of countries on a geographical map, such that two neighboring countries, i.e., countries sharing a common border, are colored differently.

Vertex Coloring (1/2)

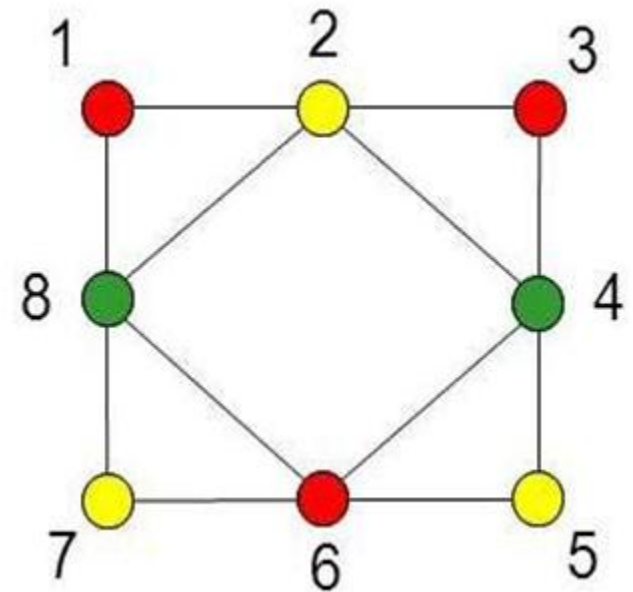
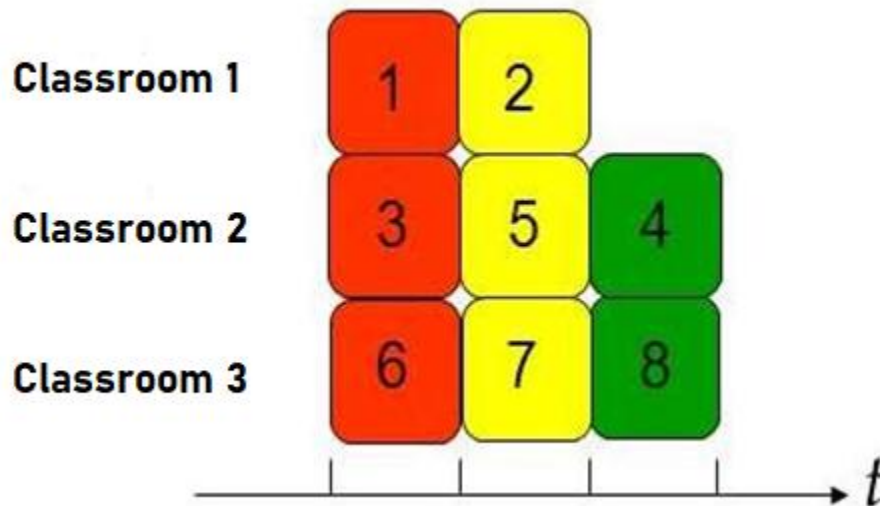
A **vertex coloring** of a graph G is an assignment of colors to the vertices such that the two endpoints of each edge have different colors. The goal is often to determine the coloring that uses the fewest possible colors.

The **chromatic number** of a graph G , denoted $\chi(G)$, refers to the minimum number of colors required to color its vertices.

A graph is said to be **p -chromatic** if its vertices can be colored with exactly p colors.

Vertex Coloring (2/2)

Example: Achieving optimal time management is a significant challenge, as it requires strict organization, careful planning, and the ability to prioritize tasks effectively.



Edge Coloring

An "**edge coloring**" of a graph **G** is an assignment of colors to the edges such that any two edges sharing a common vertex are assigned different colors. The goal is usually to determine a coloring that uses the fewest possible colors.

The minimum number of colors required to color the edges of a graph **G** is called the "**chromatic index**" of **G** and is denoted $\chi'(G)$.

Propositions (1/2)

Property 1: The chromatic number of the complete graph K_n is n .

Property 2: If a graph G contains a complete subgraph of order n , then the chromatic number of G is **at least n** .

Property 3: The chromatic number of a graph is **at most the maximum degree of its vertices**.

Property 4: If a graph G contains a cycle of chromatic number k , then the chromatic number of G is **at least k** .

Propositions (2/2)

Brooks' theorem : Let G be a connected, undirected graph, and let be r the maximum degree of any vertex in G . Then the chromatic number $\chi(G)$ of G satisfies the following conditions:
If G is neither a complete graph nor an odd cycle, then $\chi(G) \leq r$.

If G is a complete graph or an odd cycle, then $\chi(G) = r+1$.

Berge' theorem(1970): The chromatic index of a bipartite graph G is equal to r , where r is the maximum degree of any vertex in the graph G .

Coloring Algorithms: Greedy Algorithms

Greedy algorithms are commonly used to solve optimization problems. The principle of such algorithms is to iteratively choose locally optimal solutions with the aim of obtaining a globally optimal solution.

In the case of **Graph Coloring**, this involves coloring the vertices one by one, assigning each vertex the smallest possible color that is not already used by any of its neighbors. The order in which the vertices are processed defines different variants of the algorithm. One method is to color the vertices in decreasing order of their degree.

Principle of the Welsh-Powell Algorithm

- a. **Vertex Sorting:** The vertices of the graph are first sorted by **decreasing degree**. The idea is to start coloring the vertices with the highest number of neighbors, as these vertices are more constrained in their choice of color.
- b. **Coloring:** After sorting, the algorithm assigns a color to each vertex. For each vertex, it chooses the **color available**, i.e., a color that has not yet been assigned to any of its already colored neighbors.
- c. **Repetition:** This process (**step b**) is repeated for all vertices in the graph until all vertices are colored.

Welsh-Powell Algorithm Steps

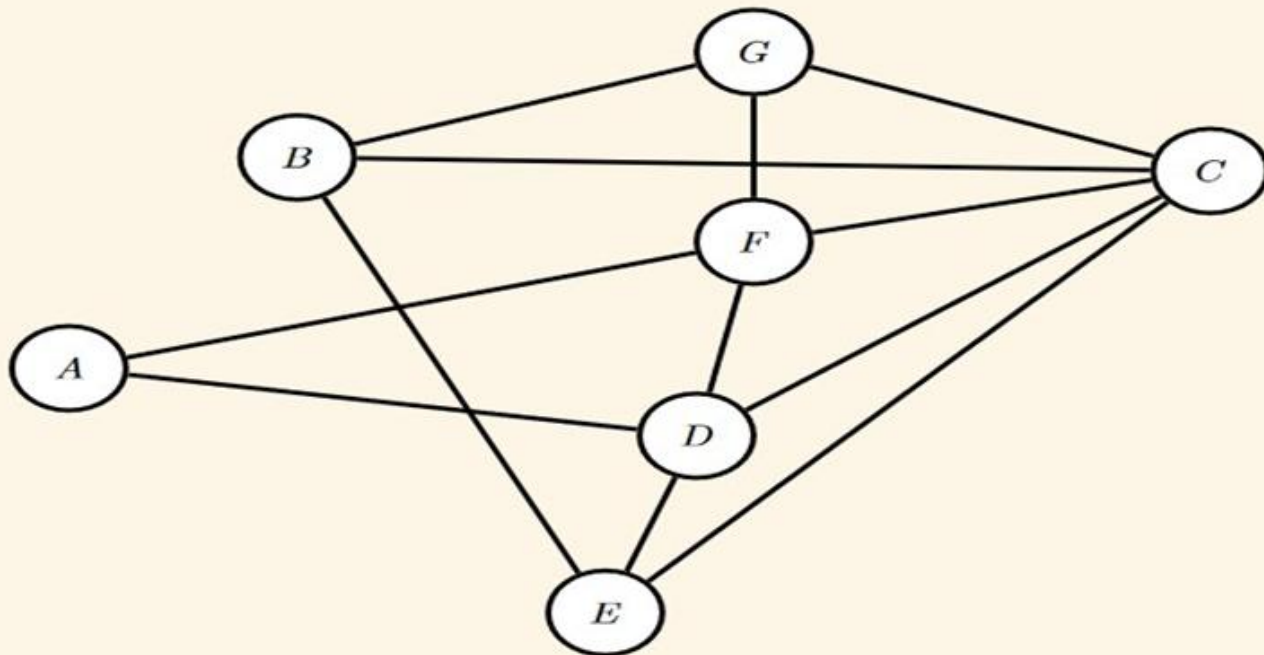
1. Identify the degree of each vertex.
2. Sort the vertices in decreasing order of their degree.
3. Assign a color to the first vertex (A) in the list.
4. Traverse the list and assign the same color to the first vertex (B) that is not adjacent to (A).
5. Continue traversing the list and assign the same color to the next vertex (C) that is not adjacent to any previously colored vertices (A or B).
6. Continue until all vertices in the list have been processed.
7. Choose a second color for the first vertex (D) that has not yet been colored.
8. Repeat steps 4 to 7 until all vertices are colored.

Welsh-Powell Greedy Algorithm (3/12)

Example

The vertices can be ranked by decreasing degree, and then colored.

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

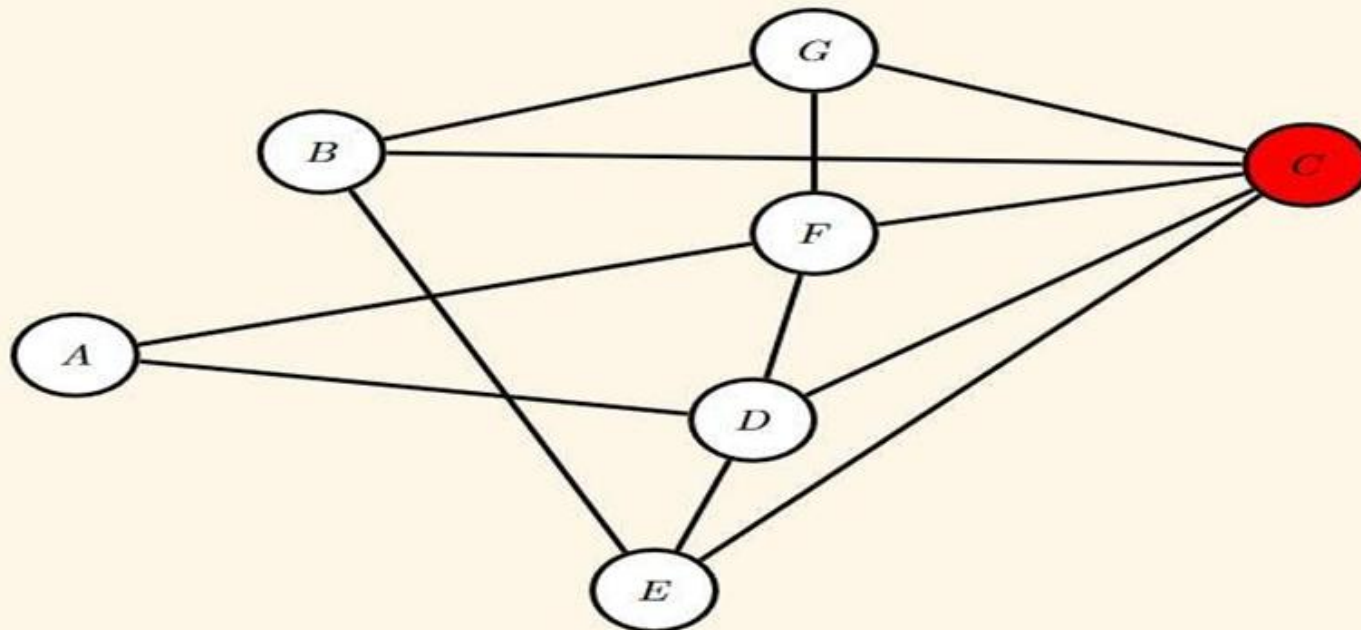


Welsh-Powell Greedy Algorithm (4/12)

Example

One possible coloring could be: Color **C** in red color, then proceed in decreasing order...

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

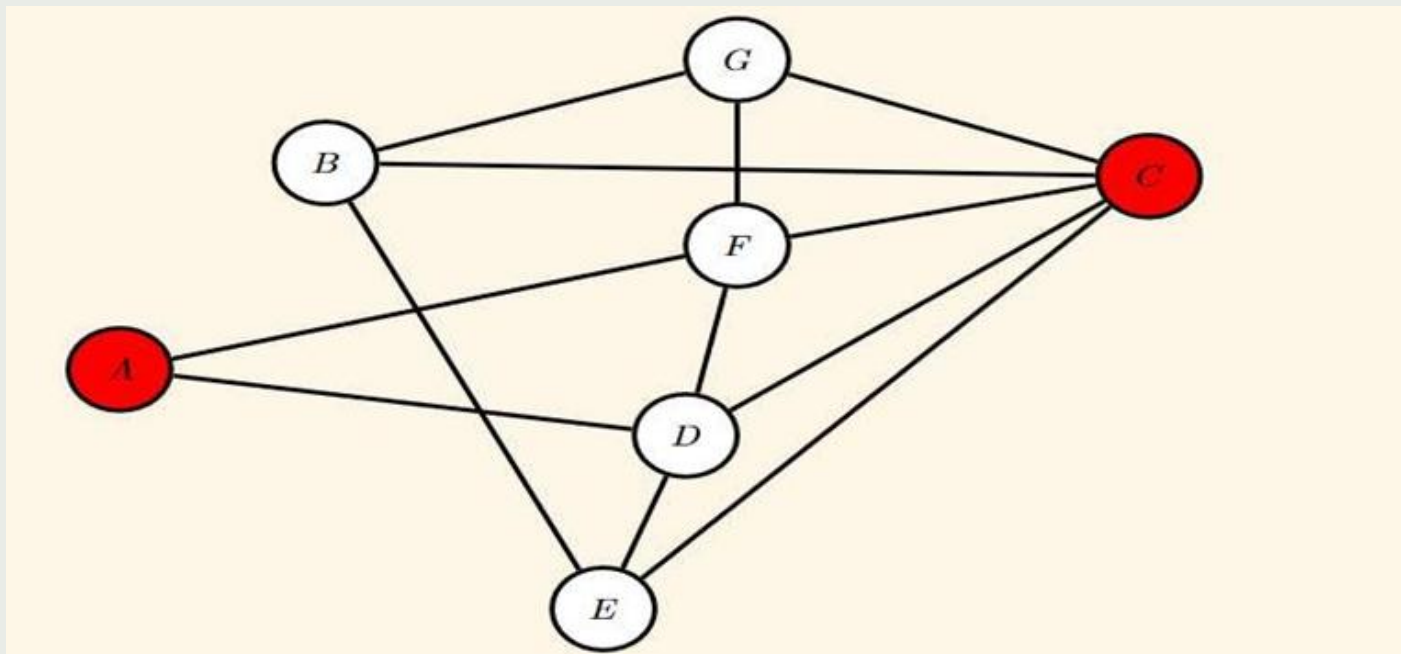


Welsh-Powell Greedy Algorithm (5/12)

Example

Color the non adjacent vertex **A** in red color.

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

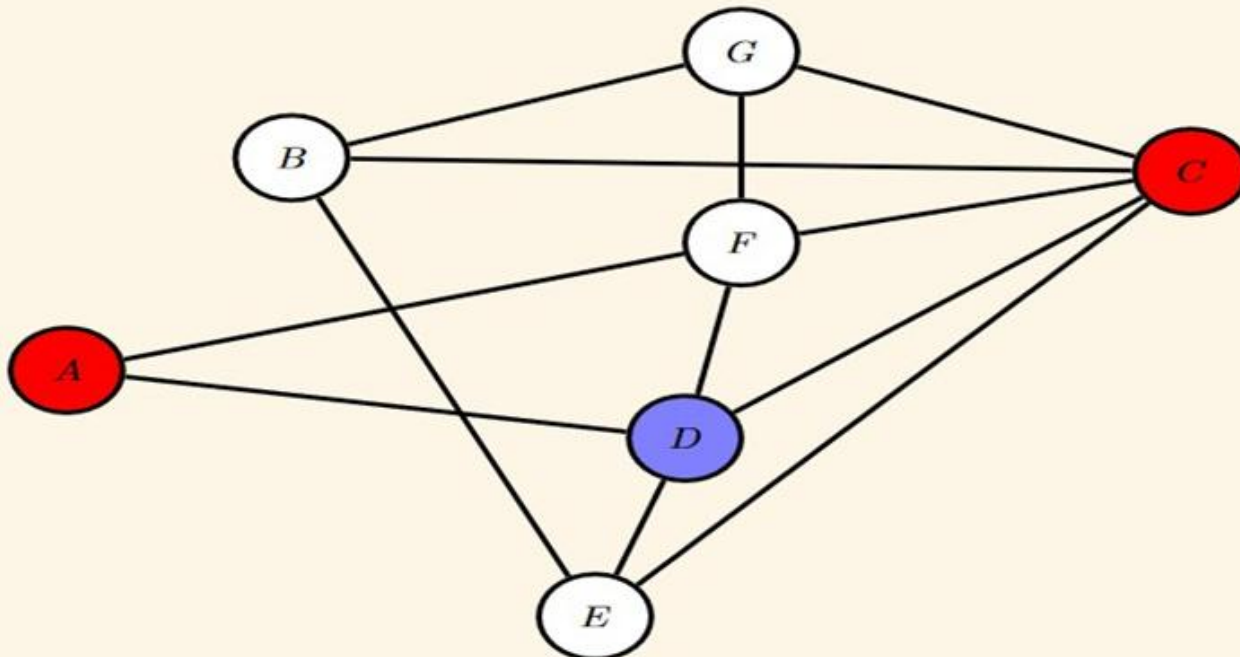


Welsh-Powell Greedy Algorithm (6/12)

Example

Then we change the color, color **D** in blue color, then proceed in decreasing order....

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

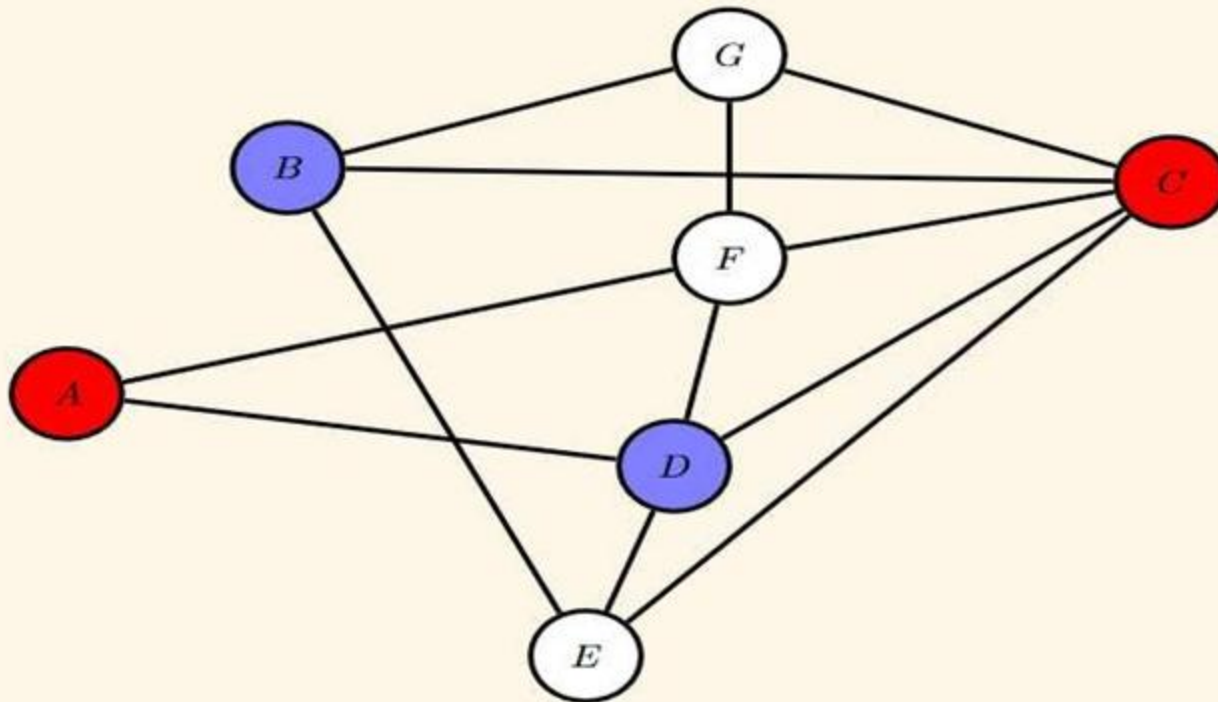


Welsh-Powell Greedy Algorithm (7/12)

Example

Color the non adjacent vertex **B** in blue color.

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

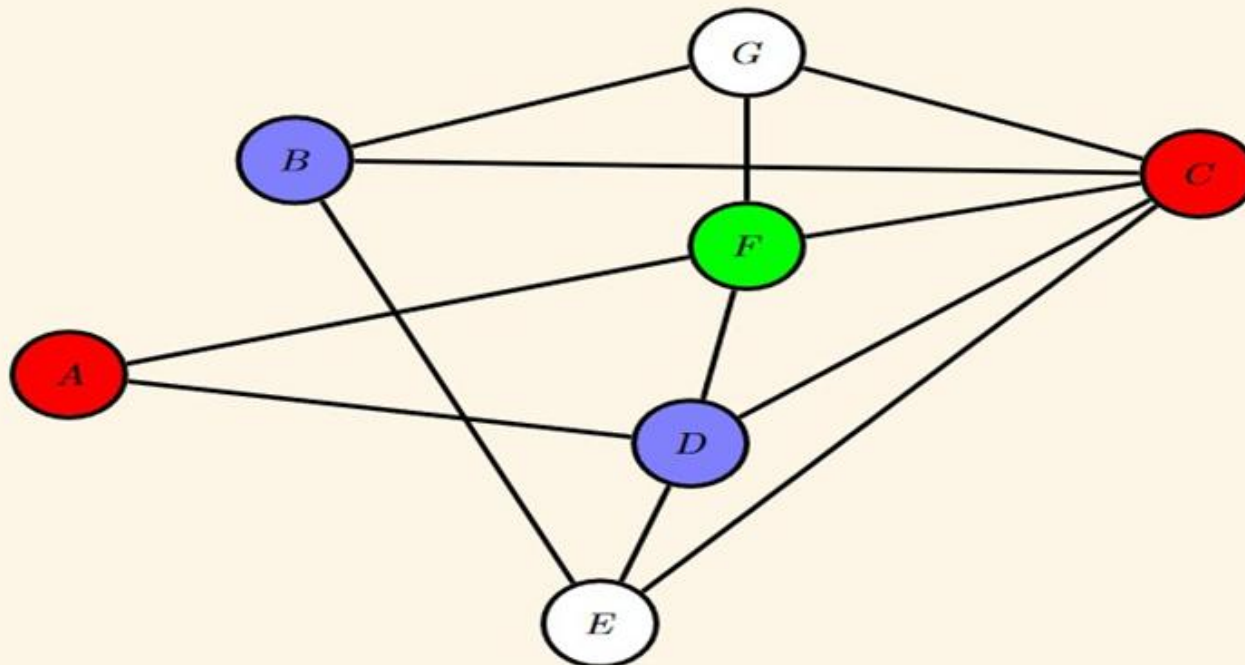


Welsh-Powell Greedy Algorithm (8/12)

Example

Then we change the color, color **F** in green color, then proceed in decreasing order....

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

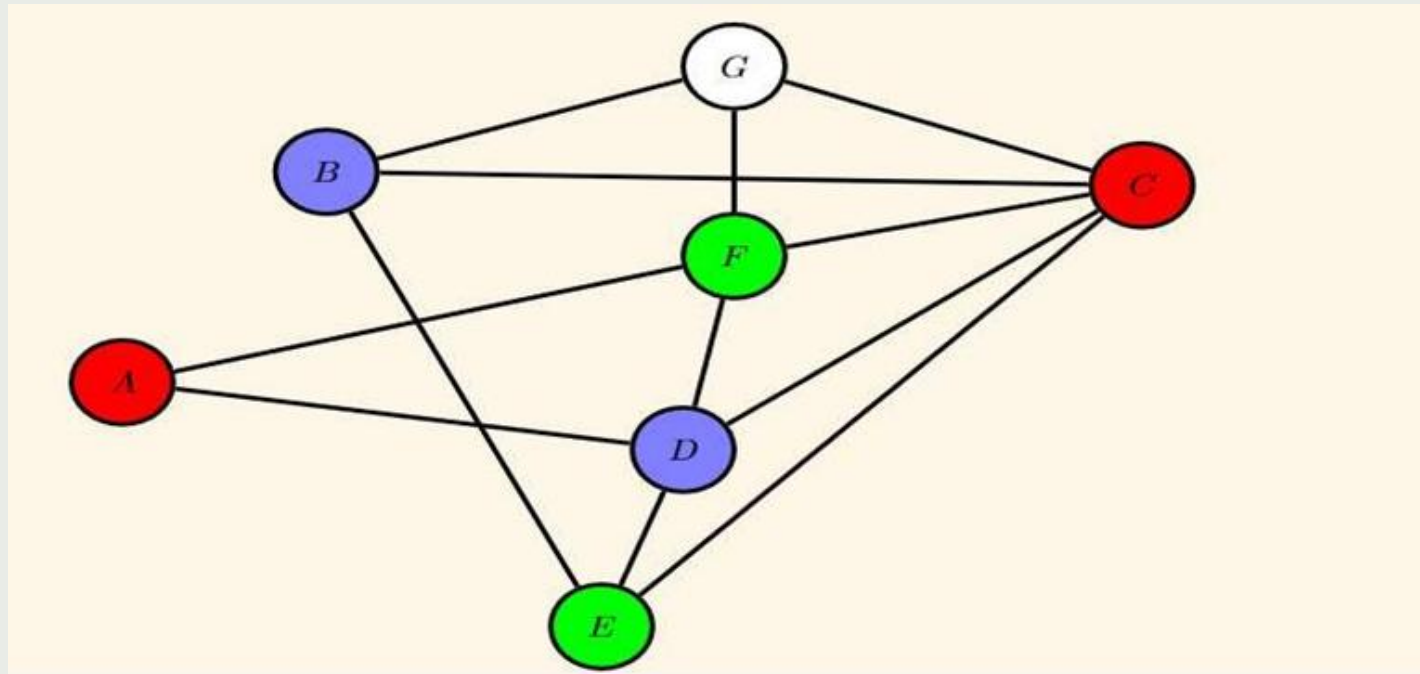


Welsh-Powell Greedy Algorithm (9/12)

Example

Color the non adjacent vertex **E** in green color.

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2

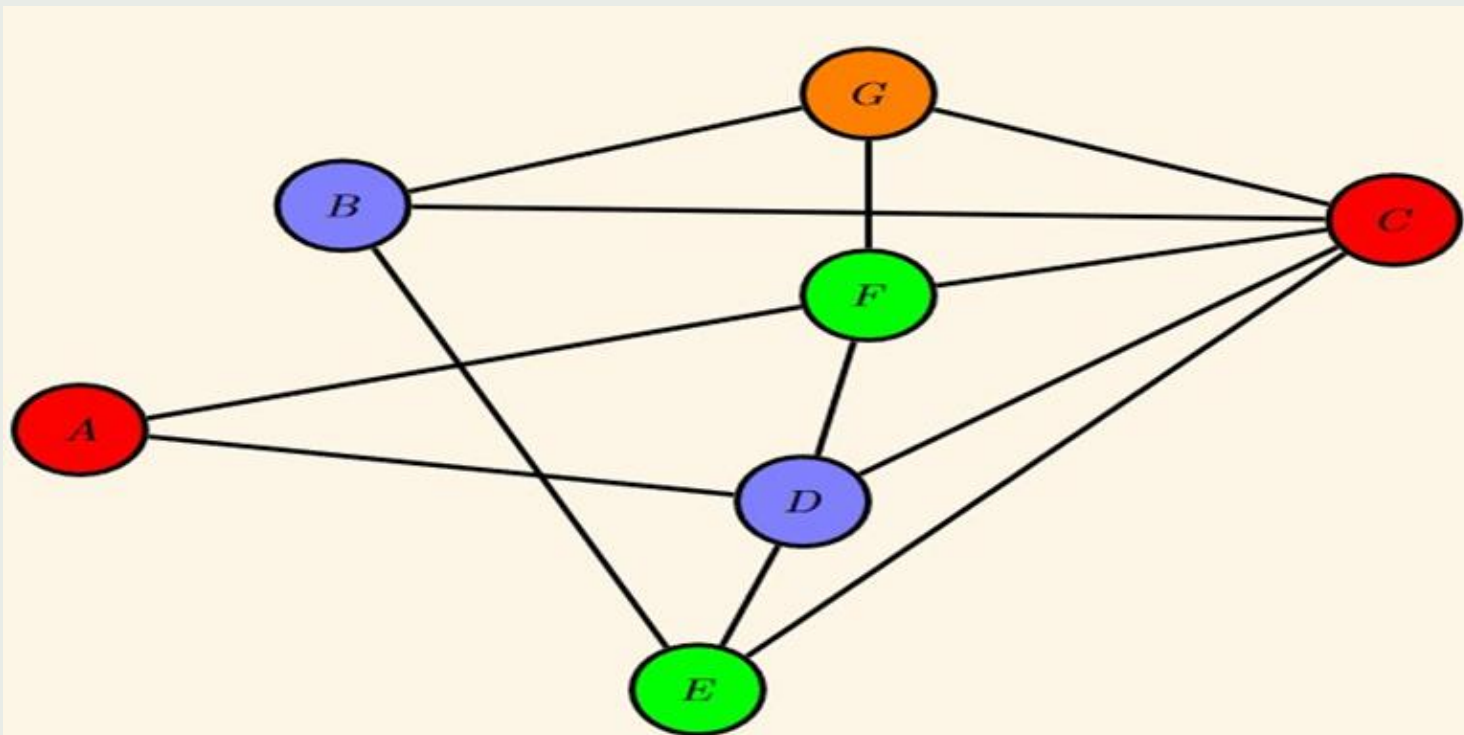


Welsh-Powell Greedy Algorithm (10/12)

Example

Finally, we change the color, color **G** in orange color.

Vertex	C	D	F	B	E	G	A
Degree	5	4	4	3	3	3	2



Advantages

- **Simplicity and efficiency:** Easy to implement and runs quickly for many graphs.
- **Reasonable computation time:** efficient for graphs of moderate size.
- **Good performance in some cases:** Provides a fast and effective approximate solution, especially for graphs with average or low degree.

Disadvantages

- **Non-optimality:** It does not guarantee an optimal solution in terms of the number of colors and may use more colors than necessary.
- **Dependency on vertex order:** The result may vary depending on the order in which the vertices are processed.
- **Not suited for dense graphs:** Less effective for graphs with many edges.
- **Approximate results:** Does not always provide the best possible solution.

Principle of DSATUR Algorithm

- **DSATUR** is a graph coloring algorithm created by Daniel Brélaz in 1979 at EPFL (École Polytechnique Fédérale de Lausanne).
- It is a sequential coloring algorithm based on heuristics.
- **DSATUR** is an abbreviation for "**d**egree of **s**atur**ation**" and is calculated as follows:
 - a. If no neighbor of vertex **x** is colored, then $\text{DSATUR}(x) = 0$.
 - b. Otherwise, $\text{DSATUR}(x)$ = the number of different colors used in the neighborhood of **x**.

DSATUR Algorithm Steps

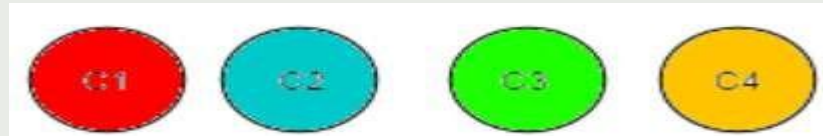
At the beginning, the graph is uncolored.

1. Order the vertices in decreasing order of degree.
2. Color a vertex with the maximum degree using color 1.
3. Choose an uncolored vertex with the **maximum DSATUR**.
4. In case of a equality, choose the one with the maximum degree.
5. Color this vertex with the smallest available color.
6. If all vertices are colored, then stop. Otherwise, go back to step 3.

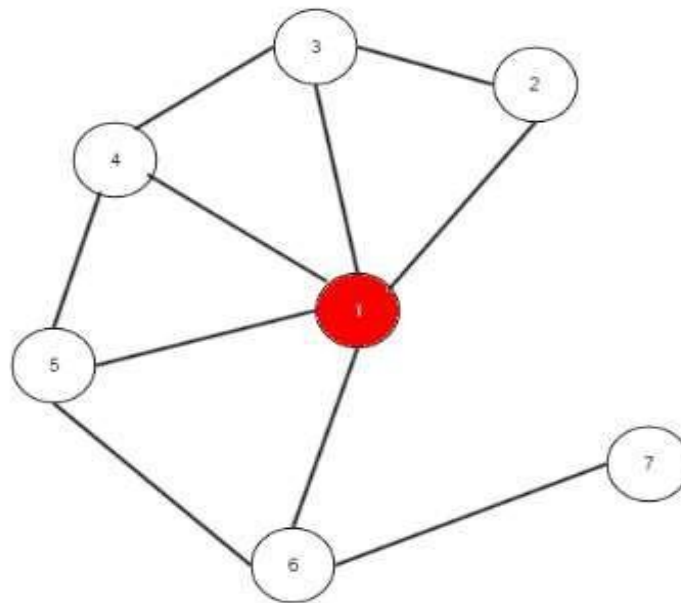
DSATUR Algorithm (3/6)

Example

Let the colors be hierarchized:

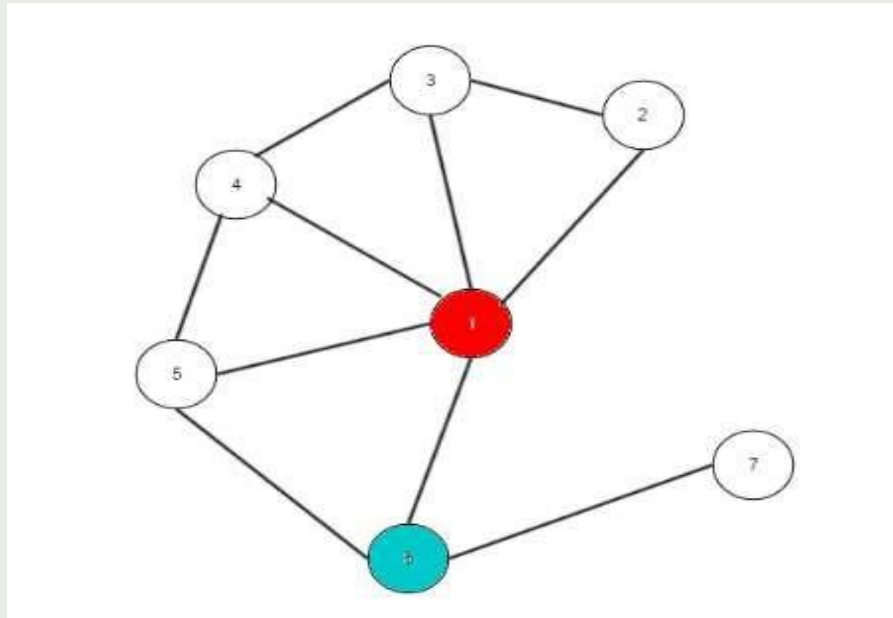


We take the vertex with the highest degree, which is vertex 1.
We assign it the minimum color, i.e., C1.



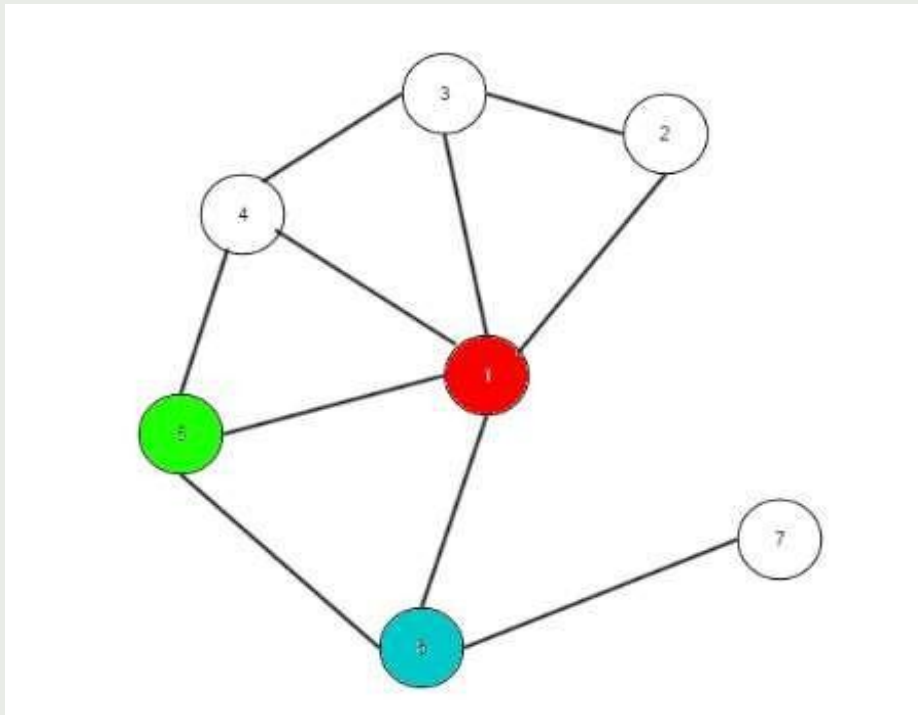
Example

Then, we examine the neighbors of vertex **1**, starting with the one having the **highest DSATUR**. In case of equality, we select the one with the highest degree. We select vertex **6** and assign it the smallest available color, but different from the color of vertex **1**. We choose **C2**.



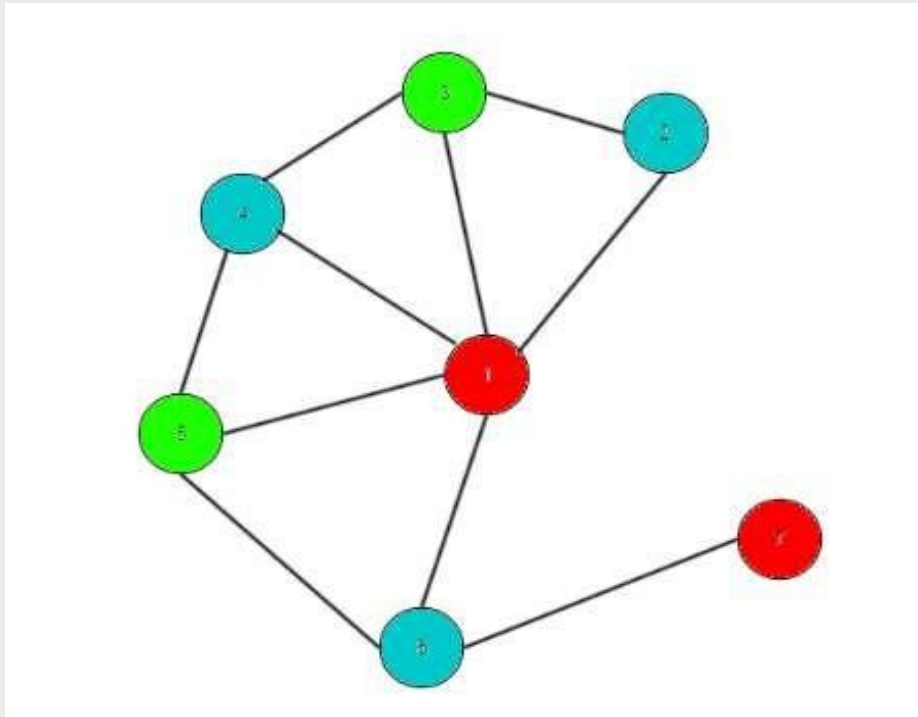
Example

We apply the same procedure for the neighbor of the vertex with the **highest DSATUR**, namely vertex **5**. We assign it the smallest available color, different from that of its neighbors: color **C3**.



Example

We repeat the process for all the other vertices. In the end, we obtain:



The Four Color Problem

One of the most famous problems in graph theory is the **Four Color Problem**. This problem remained unsolved for more than a century. In **1852**, **F. Guthrie** asked whether it was possible to color a geographical map using **four colors** such that neighboring countries have different colors.

Numerous works were published in the following century to reduce the number of colors to four, until the final proof by American researchers **K. Appel** and **W. Haken** in **1976**. They were able to affirmatively answer this **Four Color Conjecture**.

The Four Color Theorem

Theorem: Every planar graph is 4-chromatic.

Application :

It is possible to color any geographical map with four colors so that two regions sharing a common border have different colors.